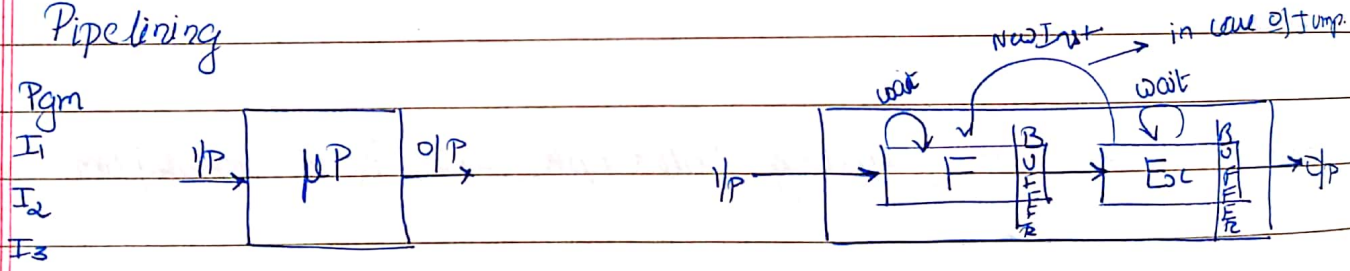
10/02/20
Lect

Pipelining



Pipelining divides instruction cycle into different stages. Each stage has a different circuitry. New inputs are accepted at one end before previously o/p occurs at other end

	T_1	T_2	T_3
I_1	F	EXC	
I_2	I	F	EXC
I_3			F

Ins. Prefetch

Scanned by CamScanner

Execute takes longer than fetch, wait is introduced and a buffer is needed.
For jump statements all pre-fetched instructions must be flushed and new inst. must be fetched, exec. circuitry must wait

Typical 6-stage pipeline

FI DI Calculate operand (EA) Fetch op. Exc. Acc+reg. Write op.

Factor speed owed to minimal wait

Consider a program having 9 instructions, each inst has 6 stages. How much time will be taken if this program is exec. in a non-pipelined proc., time in pipelined proc.
Assumptions: Each stage : 1 unit of time, each inst undergoes all 6 stages

I_1: 6 ~~st~~ time units I_2: 7 (+1) I_3: 8 (7+1)

$I + Stage - 1 = 14 \times Time = 14$ units

Non-Pipe. $= 9 \times 6 = 54 \times Time = 54$ units.

Pipelined time : $[Stage + (No. of inst. - 1)] \times Time$

Inst I_3 leads to a conditional branch to I_{15} (branch occurs)

	T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
I_1	FI	DI	CO	FO	Ex	WO		
I_2		FI	DI	CO	FO	Exc	WO	
I_3			FI	DI	CO	FO	Exc	WO – Jump at 8th but since pipelined fetch of 15 occurs at T8
I_4				FI	DI			
I_5					FI	DI		
I_6						FI	DI	
⋮								
I_{15}							FI	DI CO FO Ex WO

Initially o/p at every time unit, using branch no o/p at 4 time units. Branch Penalty = 4 TU

Assumptions: Memory cannot be accessed simultaneously but some stored in cache.

Q) Assume a pip. with 4 stages, FI DA FO EX, Draw TD for a seq. of 7 inst in which inst I_{11} takes a cond. branch to I_{12}, when branch happens I_{12}-I_{15} are executed. Calculate total time in execution of program.

$(4+4-1) = 10$ $\quad$ $BP = 2(I_{13}-I_{10})^{-1}$

T_{10} $\quad$ I_{12}: T_{10} T_{11} T_{12} T_{13} $\quad$ I_{13}: T_{11} T_{12} T_{13} T_{14}

I_{14}: T_{15} $\quad$ I

$BP = (T_{10}-T_7-1) = 2$

$(4+4-1) = 7$ $\quad$ I_{12}: T_8 T_9 T_{10} T_{11}

T_7 $\quad$ I_{13}: T_{12} $\quad$ I_{14}: T_{13} $\quad$ I_5: T_{14}

$BP = 2$

I_5: T_{15} T_{16} T_{17} T_{18} $\quad$ I_6: T_{19} $\quad$ I_7: T_{20}

Total Time: 18 units $\quad$ BP: 4 units.

11/02/20 Lab:

Q1) A7 H $\quad$ 1010 0111

Jump occurs to display

0101 1000

0101 1001 $\quad$ $\boxed{59H}$ at 01H

17 H $\quad$ 0001 0111 $\quad$ o/p at 01H

FFH $\quad$ 1111 1111 $\quad$ Jump occurs

0000 0000 $\quad$ 01H at 01H

Returns 2's complement of a ~~re~~ number

2) INC does not affect carry flag but affects all others. Since carry is not affected jump never occurs as CY=0 as reset carry.

ADI affects all flag.

Scanned by CamScanner

5) B F8H

C 23H

```
   1 1 1 1
   1 1 1 1 1000
   0010 0011
   1 101 1101
X  1 111 0101
```

Jump occurs.
AC=1, S=1, P=1, Cy=0, Z=0

Speedup $= \frac{ET_{old}}{ET_{new}}$

A pipelined processor has a clock rate of 2.5 GHz and executes a program with 1.5 million inst. The pipeline has 5 stages and inst are issued at a rate of 1/clock cycle. Calculate the speedup, throughput of pipelined processor. Ignore branch penalties.
no. of inst. exec / unit time.

$\tau = \frac{1}{2.5 \times 10^9}$ s $\qquad = 5 \times 1.5 \times 10^6 \times \frac{1}{2.5 \times 10^9}$ s non pipelined

Pipelined $\frac{(1.5 \times 10^6 + 4)}{2.5 \times 10^9}$ s

(MIPS)
Throughput $= \frac{1.5 \times 10^6}{ET_{new}}$

6) Consider a non-pipelined machine with 6 exec. stages of lengths. 50 ns 50 60 60 50 50. Find instruction latency. (Avg. time to complete inst.)
Time to execute 100 inst (pipeline overhead)
Suppose we introduced pipelining, 5 ns delay b/w stages. Inst latency in this case. Time to exec. 100 inst. Calculate speedup.

Non-pipelined = Inst Latency = 320 ns

Time for 100 inst = 32000 ns.

Pipelined = Inst. latency = ~~345 ns~~ 60 (max.) + 5 ns overhead = 65 ns

Time for 100 inst = (100 + 6 − 1) × 65 = 105 × 65 ns = 6825 ns

Scanned by CamScanner

* Pipeline Hazards / Bubble
  Resource / Data / Control Hazard
* Resource Hazard: Two or more inst access the same resource at same time.
* Data Hazard: Two inst. try to access same data

A = A+B ADD A,B SUB C,A

FI DI FO EX WO

FI DI IDLE FO

RAW : Read after Write (True dependency)

WAR: Write after Read (anti dependency)

2 eg.

WAW : Write after write (output dependency). Two inst want to write to same loc. and a hazard will occur when I_2 writes before I_1

* Control Hazard: When there is branch, due to branch all subsequent inst have to be discarded.

William Stallings.

Scanned by CamScanner